

Title	Jonathan Cressman Coding Standard
Description	Personal Coding Standard
Creation date	May 16, 2026
Author	Jonathan Cressman
Filename	CodeStandard.odt
Version	1.3

Table of Contents

1	Overview.....	4
2	Copyright and Attribution.....	5
3	Overall style.....	6
3.1	Line length.....	6
3.1.1	Rules.....	6
3.2	comments.....	6
3.3	White Space and Braces.....	6
3.3.1	The Rules.....	6
3.3.2	Bracing style Example.....	7
3.4	Variable and Token General naming guideline.....	8
3.4.1	Begin with a lowercase.....	8
3.4.2	Begin with a Capital.....	8
4	Header Files.....	9
4.1	Reasons For These Rules.....	9
4.2	The Rules.....	9
4.3	Intended use of header files.....	9
4.4	Heading-by-heading Description.....	10
4.4.1	Example.....	11
4.4.2	Template.....	12
5	Module Files (.c).....	14
5.1	Reasons For These Rules.....	14
5.2	The Rules.....	14
5.3	Heading-by-heading Description.....	14
5.3.1	Example.....	16
5.3.2	Template.....	16
6	Functions.....	18
6.1	Reasons For These Rules.....	18
6.1.1	Coupling.....	18
6.1.1.1	Cohesion.....	18
6.1.2	Scope of Control and Scope of Effect.....	18
6.1.3	Pure Functions.....	18
6.1.4	Synchronous vs Asynchronous functions.....	19
6.2	The Rules.....	19
7	Function headers.....	20
7.1	Reasons For These Rules.....	20
7.2	The Rules.....	20
7.2.1	Heading-by-heading description.....	20
7.2.2	Example.....	22
7.2.3	Template.....	24
8	Variables.....	25
8.1	Reasons For These Rules.....	25
8.2	The Rules.....	25

8.3	Prefixes.....	25
8.3.1	Example.....	25
9	Redundant Names/Wrappers.....	26
9.1	Examples of things to avoid.....	26
10	Enumerated types.....	27
10.1	Reasons For These Rules.....	27
10.2	The Rules.....	27
10.2.1	Example.....	27
11	Structures and Unions.....	28
12	goto, Break, continue, resume.....	29
12.1	Return code Error Handling.....	29
12.2	Goto for handling initialization errors.....	29
12.3	Goto for Breaking out of nested loops.....	30
12.4	Break, Continue, and Resume.....	31
13	?:.....	32
14	Preprocessor.....	33
14.1	#define CONSTANT value.....	33
14.1.1	The rules.....	33
14.2	Macros.....	33
14.2.1.1	Reasons For These Rules.....	33
14.2.1.2	The Rules.....	33
14.2.1.2.1	Example.....	33
14.3	#ifdef.....	34
14.3.1	Header Files.....	34
14.3.1.1	The Rules.....	34
14.3.2	All other places.....	34
15	Comments.....	35
15.1	Reasons For These Rules.....	35
15.2	The Rules.....	35
16	Initialization, Compile time vs Run Time.....	36
17	Concurrency.....	37
18	Miscellaneous.....	38
18.1	Fall through switch case.....	38
18.2	File Naming.....	38
18.3	integer sizes.....	38
18.4	Leftover Debug Print.....	38
18.5	Return Statements.....	38

1 Overview

This document outlines the Coding conventions used by Jonathan Cressman. Its purpose is to ensure you and others **know what the code is doing**. This makes the code more readable, maintainable and error-free code. It can be modified to be a super set of conventions to be used in contract work for other companies. Many companies already have conventions about white space, token naming and brackets. When these conflict the contracting companies conventions take precedent.

This document is written with C/C++ code in mind, however it is easily transferred to other languages.

All the examples in this document are written in C/C++. The templates are also written in C/C++.

This document is not meant to be a straitjacket. It is a guide. You can break any rules in this document if you want but you must be prepared to defend your breaking of the rules in a code review. “I don't agree with the code standard”, or “I don't like that style” are not good reasons not to break the rules.

These guidelines are designed to be easy to follow and easy to keep comments and documentation up to date. Most of the time you can and should follow the rules but some times it makes your code easier to follow if you don't follow the rules 100%. You should have a good reason to not follow the rules though.

2 Copyright And Attribution

Copyright © 2026 by Jonathan Cressman is licensed under a [CC BY 4.0](#)

This work is adapted from Paul Hutchinson Coding spec v1.3 by Paul Hutchinson. Used under CC BY 4.0

Paul Hutchinson Coding spec v1.3 derived from Paul Hutchinson Coding spec v 1.0 by Paul Hutchinson, Stefan Scherer, and Dave Eccleston. Used under CC BY 4.0

3 Overall Style

3.1 Line Length

Reasons for these rules

To make code easier to work with and equally important easy for someone to read someone else's code. This section describes things that apply to all code.

Note: While modern monitors can display well over 150 characters, many tools like debug windows cannot.

3.1.1 Rules

- Lines of code before comments should not exceed 80 characters, comments should not extend the line too far
- Spaces must be used for indenting not tabs.
- The indent size will be set at 3 spaces.
- Do not use a proportional font. Use a fixed width font like Courier New.
- End of lines should match the platform the code gets used (compiled) on. CR/LF on windows, LF on Unix.

3.2 Comments

Comments should describe what the code does. The code shows **how** it is done. If a comment describes how you do something it is likely redundant. If you did something different or for security reasons you also need to comment **why**.

3.3 White Space And Braces

Reasons for these rules

To make blocks of code more recognizable.

To make following loops and conditional statements easier.

To prevent bugs occurring when code is inserted into the wrong block of code.

3.3.1 The Rules

- A brace is to appear on its own line and at the same indentation level as the block it is wrapping.
- One statement per line. For example if (x==2) x=3; should be split into two lines (one for the line, one for the then block).

- Double indent a line that has been split. If you have to continue a statement across two or more lines double indent the continued line
- All code blocks must be braced. This includes a single statement following an if.
- Whenever possible you should make the files of the project modular. You should make one file for each part of functionally.
- use blank lines to create blocks of code that complete small parts of the larger function
- cases within a switch statement are to be braced
- a comment must be added to a closing brace of an indented brace if the opening brace is more than half a page above it.
- a space shall follow an **if**, **for** or **while** key word (these are not functions)
- a double space shall precede a bracketed assignment within an if statement – see error handling
- a space shall follow a comma in any list
- a space shall precede and follow a binary operator (such as + and *)
- no space shall follow a function name and the opening brace (e.g. function())
- a space shall follow the variable type in a variable declaration of type pointer i.e. **int** ***numberPtr** not **int* numberPtr**

3.3.2 Bracing Style Example

```
#include <stdio.h>

void code(int lateHours)
{
    int Loop;          // A loop counter
    for(Loop=0;Loop<10;Loop++)
    {
        if(Loop==1)
        {
            printf("testing needed\n");
        }
        else if(Loop<5)
        {
            printf("Looping is continuing\n");
        }
        else
        {
            printf("Almost done\n");
        }
    }

    switch (lateHours)
    {
        case 0:
        {
            printf("made it\n");
        }
    }
}
```

```

        break;
    }

    case 1:
    {
        printf("do dishes\n");
        break;
    }

    case 2:
    {
        printf("bring flowers\n");
        break;
    }

    default:
    {
        printf("don't come home\n");
        break;
    }
} //switch (lateHours)
}

```

3.4 Variable And Token General Naming Guideline

Each subsequent word in a token is capitalized.

3.4.1 Begin With A Lowercase

- Functions
- Variables even if constant
- Arrays

3.4.2 Begin With A Capital

- #define Constants
- Macros
- Structs/Unions
- Enums

4 Header Files

4.1 Reasons For These Rules

In large projects the header files must provide a clear description of the modules' interfaces in order for the code to remain maintainable. Header files are a promise of an interface. They should be fairly static.

4.2 The Rules

- All header files must contain the header comment as shown in the following template.
- The header file must include all those header files necessary; but not more than are necessary to use the module.
- The body of the file should follow the order indicated in the example. If any of the sections shown are not needed the comment for that section should remain for future modifications to the interface.
- Only use `/* */` style comments in header files
- Should contain only the information necessary to use their corresponding module as the interface design intended. All other information belongs in the module file. The header file is a promise of the modules interfaces. It should remain fairly static.
- Must include any other header files needed to define external types, macros, variables, and functions needed to use the module. Not including these files will leave the programmer guessing what files are needed and the order in which unit files can be included may become non-trivial. It should not contain header files that are only needed by the implementation of the module.
- Header files do not contain the detailed function comment header. The function header goes in the module file. This is to reduce the likelihood that the function header falls out of date when the code changes.
- For complex modules or portable libraries that require the user to implement functions. If there are more than a few functions that the user must implement put these functions and comments describing their requirements in a separate header file.

4.3 Intended Use Of Header Files

Header files are to be self contained interface descriptions to their module.

Consider for example a module "serial.c" which uses modules "crc.c" and "queue.c" in its implementation. The fact that "serial.c" uses these modules should be transparent to a developer wanting to use the services that "serial.c" provides. All a developer should have to read to be able to use "serial.c" is the header "serial.h" and all a developer should have to do to use the services of "serial.h" in a module is to include "serial.h".

So "serial.c" uses "crc.c" and "queue.c" and new module, say "new.c", uses "serial.c".

If the developer now decides to also use services provided by "queue.c" the header "queue.h" must be include in the module "new.c" because the module now depend on "queue.c" directly, although "serial.h" may well have included "queue.h".

The reason for this is to hide the fact that "serial.h" is using "queue.h" and if the code for "serial.c" was changed to use "betterQ.c" your code would still be functional.

4.4 Heading-by-heading Description

Start of header

This is a line that starts with "/******"

COPYRIGHT:

This is where the copy right for this code is placed.

FILENAME:

This is the name of the file with .h on the end. This is on the next line after "FILENAME:"

PROJECT:

This is the name of the project. This is on next line after "PROJECT:"

FILE DESCRIPTION:

This is a description of what's in this file. This starts on the next line under the "E" in FILE.

CREATED BY:

This is who made this file.

It must follow the format:

Firstname Lastname (dd Mmm yyyy)

It begins on the next line under the "A" in "CREATED"

End of header

This is a normal C "*/"

HEADER FILES TO INCLUDE

List all the needed units build this header file.

DEFINES

This is where you put all non macro define's

MACROS

This is where you define an externally available macros or templates

TYPE DEFINITIONS

This is where you put your type definitions. This would be things like struct's, enums, and unions and typedef.

CLASS DEFINITIONS

This is where any externally available classes would be placed.

GLOBAL VARIABLE DEFINITIONS

This is where you place externs to any vars that are declared in the module .c file that you want others to have access to. DO NOT EXTERN VARS FROM OTHER FILES HERE!

It is of course best not to use globals at all.

EXTERNAL FUNCTION PROTOTYPES

This is where you place function prototypes that others can call.

4.4.1 Example

```
/******  
* COPYRIGHT:  
*   Copyright 2007 by Make me rich inc.  
*  
* FILENAME:  
*   financial.h  
*  
* PROJECT:  
*   Money Maker  
*  
* FILE DESCRIPTION:  
*   This is the .h file for the .c file financial.c. It does .h file stuff.  
*  
* CREATED BY:  
*   Paul Hutchinson (30 Jul 2001)
```

```

*
*****/
#ifndef __FINANCIAL_H_
#define __FINANCIAL_H_

/** HEADER FILES TO INCLUDE          */
#include "types.h"
#include "financial/misc.h"

/** DEFINES                          */
/* StealMoney() tags */
#define TAG_SM_CANADIAN    1
#define TAG_SM_BANK        2
#define TAG_SM_BESLOPPY    3

/** MACROS                          */

/** TYPE DEFINITIONS                */
typedef enum bankType_enum
{
    bank_FEDERAL_RESERVE_BANK=0,
    bank_BANK_OF_CANADA,
    bank_TD,
    bank_CIBC,
    bank_MAX
} bankType_t;

/** CLASS DEFINITIONS                */

/** GLOBAL VARIABLE DEFINITIONS      */

/** EXTERNAL FUNCTION PROTOTYPES     */
char StealMoney(char *UserName,uint16 Amount,struct accountSpec *destAccount,
                struct falseTrail *falseTrail);

#endif /* end of "#ifndef __FINANCIAL_H_" */

```

4.4.2 Template

```

/*****
* COPYRIGHT:
*
*
* FILENAME:
*
*
* PROJECT:
*
*
* FILE DESCRIPTION:
*
*
* CREATED BY:
*

```

```

*
*****/
#ifndef __ _H_
#define __ _H_

/**/ HEADER FILES TO INCLUDE      ***/
/**/ DEFINES                      ***/
/**/ MACROS                      ***/
/**/ TYPE DEFINITIONS            ***/
/**/ GLOBAL VARIABLE DEFINITIONS ***/
/**/ EXTERNAL FUNCTION PROTOTYPES ***/

#endif /* end of "#ifndef __ _H_" */

```

5 Module Files (.c)

5.1 Reasons For These Rules

The following rules are intended to provide more information to developer where it is most convenient and most easily found. The rules are to help enforce a clean interface design and provide a good framework.

5.2 The Rules

- All module files must contain the header comment as shown in the following template.
- The module file must include all those header files necessary; but not more than are necessary.
- The body of the file should follow the order indicated in the template. If any of the sections shown are not needed the comment for that section should remain for future modifications.

5.3 Heading-by-heading Description

Start of header

This is a line that starts with `"/*****"`

COPYRIGHT:

This is where the copy right for this code is placed.

FILENAME:

This is the name of the file with `.c` on the end. This is on the next line following `"FILENAME:"`

PROJECT:

This is the name of the project. This is on the next line following `"PROJECT:"`

FILE DESCRIPTION:

This is a description of what's in this file. This starts on the next line under the `"E"` in `FILE`.

CREATED BY:

This is who made this file.

It must follow the format:

Firstname Lastname (Month day yyyy)

It begins on the next line under the "A" in "CREATED"

End of header

This is a normal C "*/"

HEADER FILES TO INCLUDE

List all the unit's that you need to build this module.

DEFINES

This is where you put all non macro local define's

MACROS

This is where you define local macros

TYPE DEFINITIONS

This is where you put struct's, enums, and unions. DO NOT declare variables here. i.e. do not:

```
/** TYPE DEFINITIONS      */
```

```
struct myStuff
```

```
{
```

```
    int x;
```

```
} myVar;
```

Instead declare the struct here, and then the var in the "VARIABLE DEFINITIONS" section. Ie.

```
/** TYPE DEFINITIONS      */
```

```
struct myStuff_s
```

```
{
```

```
    int x;
```

```
};
```

```
/** VARIABLE DEFINITIONS  */
```

```
struct myStuff_s myVar;
```

VARIABLE DEFINITIONS

This is where you place any vars that are in your .c file that you want others to have access to. DO

NOT EXTERN VARS FROM OTHER FILES HERE!

FUNCTION PROTOTYPES

This is where you place function prototypes that are only used by this file.

5.3.1 Example

```
/* *****
 * COPYRIGHT:
 *   Copyright 2007 by Make me rich inc.
 *
 * FILENAME:
 *   financial.c
 *
 * PROJECT:
 *   Money Maker
 *
 * FILE DESCRIPTION:
 *   This file has functions for accessing financial systems (like banks).
 *
 *
 * CREATED BY:
 *   Paul Hutchinson (30 Jul 2001)
 *
 * *****/

/** HEADER FILES TO INCLUDE */
#include "types.h"
#include "financial/taxshelter.h"
#include "financial/errors.h"

/** DEFINES */
/* DestoryEvidence() flags */
#define DE_QUICK      (1L<<0)
#define DE_FORCE      (1L<<1)
#define DE_BACKTRACK (1L<<2)

/** MACROS */

/** TYPE DEFINITIONS */

/** VARIABLE DEFINITIONS */

/** FUNCTION PROTOTYPES */
void destoryEvidence(char *userName, u_int32_t flags);
```

5.3.2 Template

```
/* *****
```

```

* COPYRIGHT:
*
*
* FILENAME:
*
* PROJECT:
*
*
* FILE DESCRIPTION:
*
*
* CREATED BY:
*
*
*****/

/**** HEADER FILES TO INCLUDE ****/

/**** DEFINES ****/

/**** MACROS ****/

/**** TYPE DEFINITIONS ****/

/**** VARIABLE DEFINITIONS ****/

/**** FUNCTION PROTOTYPES ****/

/**** FUNCTIONS ****/

```

6 Functions

6.1 Reasons For These Rules

These rules are to make reading functions easier and to reduce the scope of any bugs.

When reducing a problem down into functions remember to keep in mind coupling, cohesion, scope of control and scope of effect.

6.1.1 Coupling

Coupling is the amount of dependency between functions and modules. Try to keep this to a minimum. Functions should have a well defined interface, a single entry point and a single exit.

- Data coupling – required to some extent, however functions should not contain input parameters that are switches. If a function contains an input parameter that is a switch this is a good indication that the function should be broken into a separate function for each switch or there was a mistake in the scope of effect.
- Content coupling – each module should never rely on the internal workings of another function.
- for more information on coupling [http://en.wikipedia.org/wiki/Coupling_\(computer_science\)](http://en.wikipedia.org/wiki/Coupling_(computer_science))

6.1.1.1 Cohesion

- please review [http://en.wikipedia.org/wiki/Cohesion_\(computer_science\)](http://en.wikipedia.org/wiki/Cohesion_(computer_science))

6.1.2 Scope Of Control And Scope Of Effect

Scope of control is all the functions called by a given function and all the functions called by those functions. Scope of effect (of a decision) is all the processing that is conditional on a decision. The scope of effect of a decision should be a subset of the scope of control of the module where the decision was made. A good indication there is a possible scope problem is multiple if statements for the same condition.

6.1.3 Pure Functions

A pure function is one that is deterministic based on its input variables and has no side effects. That is it doesn't use any global variables or internal static variables, it only calls pure functions and it doesn't modify any global variables. Ideally most functions will be pure functions. The excessive use of global variables is usually a sign of developers hacking code in a project they don't fully understand.

6.1.4 Synchronous Vs Asynchronous Functions

A synchronous function returns right away. It doesn't wait on anything. It doesn't spin or call any asynchronous functions. Taking a semaphore or lock makes a function asynchronous. Ideally most functions will be synchronous. The use of excessive asynchronous functions is usually as sign of poor architecture particularly the use of excessive concurrency controls.

6.2 The Rules

- Function names should be verbs.
- most function names start with a lower case letter and have the first letter of each subsequent word capitalized. Wrapper functions should have the same name as the function they wrap except start with a capital letter.
- function parameters should be listed output, input/output, input.
- A function only can have one format for its output it may accept multiple input formats
- A function should never contain a goto statement that branches out of the function

7 Function Headers

7.1 Reasons For These Rules

These rules are necessary for easy and automatic documenting of modules or libraries.

A small parsing script will be able to easily extract header information to create reference materials for other developers and customers. To aid with the automation the header formatted in a style similar to a man page.

There have also been many people point out that there is a lot of duplicated information in the header that can be figured out by looking at the real code below the header. The main reason for the duplication is “glancing”. This lets the reader (aka programmer) see parts at a glance without having to think about it (not having to decode the prototype). This speeds things up when the reader is looking for something in a big file.

The order of the headings are also important for readability and quick referencing. In English we read from the top left to the bottom right. We are use to seeing information presented this way. That is why we have the synopsis (the prototype) before the function description and directly above the parameters.

7.2 The Rules

- The function header format must be as outlined in the following templates.
- No line in the header must be longer then 100 characters.
- Optional sections can be deleted if there is no information to be entered

7.2.1 Heading-by-heading Description

Start of header

All headers must begin with an old style C comment. It must start with `"/***** "` (A forward slash followed by 6 asterisks and then a space). The header ends when the C comment ends `(*/)`.

NAME:

This is the name of the function and only the name of the function (no parameters or description). The name must start on the next line under the "E" in NAME. This is used for sorting and making lists of available functions.

SYNOPSIS:

This is an ANSI standard the prototype of the function. Remember lines can't be greater 100 chars.

PARAMETERS:

This lists each of the arguments that are sent into your function. Use type and the same name token used in the first SYNOPSIS line (so the user can match inputs to the descriptions).

List the output parameters first, then any parameters that are both input and out put and finally the input parameters. If any global variables are used or modified list them as well.

You should think of this as the data and not the parameter it's self. That is to say if you are passing in a pointer to a buffer and this is where you store your data you would list it was an output even though you are not going to change where the pointer points.

After the variable type and name pair the preferred follower is "--". (See example). After the -- comes a description of what this parameter is used for/how it is used.

If the description is longer than 80 chars you need to wrap the line. You should indent using one of the following styles:

```
input -- If the description is long, then indent the
        second line so that is just after the divider.
```

OR

```
input -- If the description is long and the argument is long, then indent the
        second line by 6 spaces.
```

These are the only 2 indenting styles.

If there are no parameters for your function then put the word "NONE" here.

The following is an example of a function that reads from an open disk file:

```
char *buffer      -- The buffer to store the read data
int fileHandle    -- The file system handle to the file to read
int size          -- The number of bytes to read
```

FUNCTION:

Describe what your function does in generally accepted English. Keep jargon to a minimum, but don't sacrifice clarity and accuracy. You may even take the radical step of using a spelling checker.

RETURNS:

Describe what your function returns. Saying it returns success is not acceptable. Describe what success means (Ie. TRUE – things worked, FALSE -- There was an error). If your function returns nothing (void) then put the word "NONE" here.

THROWS:

If your function throws error this is a list of what it can throw.

NOTES:

This is optional. Put helpful hints, warnings, tricks, traps, etc here.

BUGS:

This is optional. If there are any, describe the bug, and how it can be avoided. List versions, workarounds, etc. Remove entries from this if you fix them.

This is also where you might put limitations of your function if you don't handle every possible input case. For example if your function calculates if a point is in a box, but you can only currently do square boxes (but it is possible to expand the function).

EXAMPLE:

This is optional. A **short** example of how your function is called. This must BE TESTED. Write, test, then remove lines if needed to shorten the example. Use "..." to indicate removed sections. Do not edit the example after creation (unless you retest). C compilers do not allow nested /* */ comments. Use // Instead (remember you are inside a big /* */ pair).

SEE ALSO:

If there are other functions which help describe the data structures, or are otherwise related to this function, place their names here. Note include files, where appropriate (it is acceptable to list just the ".h" file)

Functions in THIS module are simply listed, with () to indicate they are a function. Functions from OTHER modules are preceded by the module filename. (See example)

If there is nothing that you want to list you can just leave this blank or list "NONE".

Note you do not (and should not) list every function your function calls. List only things that a user may also want to look at.

Think of this a list of grouped functions. For example fopen fclose, fread, fwrite would be part of a group, but printf doesn't fit in this group because although it does have to do with IO it doesn't have anything to do with file control (fprintf would though).

End of header

This is a normal C "*/"

7.2.2 Example

```
/*  
 * NAME:
```

```

*   stealMoney
*
* SYNOPSIS:
*   char stealMoney(char *userName, u_int16_t amount,
*       struct accountSpec *destAccount, struct falseTrail *falseTrail,
*       TAGLISTTYPE tag,...);
*
* PARAMETERS:
*   INPUT
*   char *userName -- Name to make the transaction under. Popular favorites
*                   include "Ronald Reagan" and "Mohamar Quadaffi".
*   u_int16_t Amount -- Number of dollars to transfer (in thousands).
*   struct AccountSpec *DestAccount -- A filled-in AccountSpec structure
*   detailing the destination account (see financial/accounts.h). If
*   NULL, a second Great Depression will be triggered.
*   struct FalseTrail *FalseTrail -- If the DA_FALSETRAIL bit is set in
*   'DestAccount', a 'FalseTrail' structure must be provided.
*   TAGLISTTYPE tag -- Supported tags:
*       TAG_SM_CANADIAN -- The Amount will be in Canadian funds.
*                           Defaults to FALSE. (boolean)
*       TAG_SM_BANK -- What bank to use to make the transactions.
*                           Defaults to bank_FederalReserveBank. (BankType)
*       TAG_SM_BESLOPPY -- Cut corners when stealing the money. This
*                           will leave a trail of evidence. Useful for
*                           framing people who don't logout of there
*                           terminals when they go home at night.
*                           Defaults to FALSE. (boolean)
*
* FUNCTION:
*   Transfer money from a bank into the specified interest-earning
*   checking account. No records of the transaction will be retained unless
*   the tag TAG_SM_BESLOPPY is set.
*
* RETURNS:
*   Zero for success, else an error code is returned (see financial/errors.h).
*
* NOTES:
*   Do not run on Tuesdays!
*
* BUGS:
*   Before V88, this function would occasionally print the address and
*   home phone number of the caller on local police 976 terminals.
*   We are confident that this problem has been resolved.
*
* EXAMPLE:
*   Federal regulations prohibit a demonstration of this function.
*
* SEE ALSO:
*   createAccountSpec(), destoryEvidence(), falseTrail.c/makefalsetrail(),
*   financial/misc.h
*
* CREATED
*   Paul Hutchinson (10 Jul 2001)
*****/

```

7.2.3 Template

```

/*****
* NAME:
*
*
* SYNOPSIS:
*
*
* PARAMETERS:
*   OUTPUT
*   INPUT/OUTPUT
*   INPUT
*
* GLOBAL VARIABLES
*
* FUNCTION:
*
*
* RETURNS:
*
*
* THROWS:
*
*
* NOTES:
*
*
* BUGS:
*
*
* EXAMPLE:
*
*
* SEE ALSO:
*
* CREATED
*
*****/
```

8 Variables

8.1 Reasons For These Rules

Naming and declaration of the variables should make it easy for the reader to understand the use and purpose of the variable.

8.2 The Rules

- Applicable prefixes are to be applied in the order they appear listed below.
- All prefixes are to be lower case letters so that the beginning of the actual variable name can be spotted easily.
- There is a lot of flexibility in how you name your variables. You must make it clear what the variable is for. The recommended naming style is to capitalize all the words except the first in the name and to avoid under scrolls _.
- Only declare one variable per line ,the function prototype is an expectation as are variables where their use is obvious. Simple loop counters can be considered obvious.
- Variable names should be nouns. If the variable is a measure of something the units should be in the name. e.g. timeSeconds, massKg, etc.
- If the variable is a non string pointer and is being used in any pointer arithmetic it should have the prefixed ptr_.
- function pointers shall be prefixed with ptr_
- After a variable declaration a description of what the variable does should be include.

8.3 Prefixes

- External/Global variables are to prefixed with 'g_'.
- Module/static file variables are to be declared as static and prefixed with 'm_'.
- Local/Stack/function static variable, including function parameters have no special prefix.

8.3.1 Example

```
int g_lastErrorCode;    // This is a global variable
int m_currentStep;      // This is a module variable

void fn(void)
{
    int localVar; // This is a local variable
}
```

9 Redundant Names/Wrappers

Unnecessary typedefs, enums, structures wrapper functions, and macros should be avoided whenever possible. If a structure only has one element it likely is redundant (it also means a lot of code will be used just changing between types). typedefing another datatype likely means the project has two names for the same thing, choose one. Another misuse of a type def is to describe a variable.

Do not typedef structs

9.1 Examples Of Things To Avoid

```
struct MacAddress_s
{
    uint8_t macAddress[6];
}
```

```
typedef uint8_t parseIdx_t;
parseIdx_t parseIndex;
```

10 Enumerated Types

10.1 Reasons For These Rules

These rules will insure that enumerated values are easily distinguished from variables.

You should use enumeration when a group of constants is declared, that naturally fall into a list.

The enum declaration is an aid for tagging in IDEs such as eclipse.

10.2 The Rules

- Enumerations shall be suffixed with '_enum' while their typedefs suffixed with '_t'
- Each entry in the enumeration shall be prefixed with the enumeration name and '_'. The rest of the entry shall be capitalized. Multiple words are separated by '_'. This should avoid any name space collisions while also showing that enums are constants.
- If a list is created based on the elements of an enumerated list the first entry shall be explicitly set and the after the last enumerated value the entry enumname_MAX added.

10.2.1 Example

```
typedef enum Number_enum
{
    Number_ONE=0,
    Number_TWO,
    Number_THREE,
    Number_MAX
} Number_t;
```

11 Structures And Unions

Structure and union definitions will be suffixed with `_s` or `_u`

```
struct MyStruct_s
{
    uint8_t stuffVar1;
    uint16_t stuffVar2;
}
```

```
union MyUnion_s
{
    uint8_t stuffVar1;
    uint16_t stuffVar2;
}
```

12 Goto, Break, Continue, Resume

12.1 Return Code Error Handling

It is permissible to check a return value in an if statement. For example:

```
if ( (socketFd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
{
    //some socket Creation Error Handling
}
```

Note the double space before the brackets for the assignment. The brackets around the assignment are required since the '<' less than operator has precedence over assignment.

12.2 Goto For Handling Initialization Errors

The much maligned goto statement still has a place in C programming. Consider a block of code that must initialize a number devices and allocate resources. Additionally if any of these initializations or allocations fail the code must free the resource already allocated and uninitialized any devices previously initialized before the failure.

Without a goto this could be written:

```
if ( allocateA() == ERROR)
{
    return ERROR;
}

if ( allocateB() == ERROR)
{
    freeA();

    return ERROR;
}

if ( allocateC() == ERROR)
{
    freeB()
    freeA();

    return ERROR;
}

if ( allocateD() == ERROR)
{
    freeC()
    freeA();
}
```

```
    return ERROR;
}
```

(Can you spot the error above?)
or using goto statements it could be written:

```
if (allocateA() == ERROR)
{
    goto failA;
}
```

```
if (allocateB() == ERROR)
{
    goto failB;
}
```

```
if (allocateC() == ERROR)
{
    goto failC;
}
```

```
if (allocateD() == ERROR)
{
    goto failD;
}
```

```
do something useful
```

```
return SUCCESS;
```

```
failD:
    freeC();
```

```
failC();
    freeB();
```

```
failB:
    freeA();
```

```
failA:
    return ERROR;
}
```

12.3 Goto For Breaking Out Of Nested Loops

Additional places where a goto statement might be appropriate would be for breaking out of multiple nested loops.

12.4 Break, Continue, And Resume

Break, Continue, and Resume are really disguised gotos. Mixing them in the same loop creates the danger of issues not easily spotted such as bypassing an increment/decrement or bypassing a test condition. Used in anything other than a small loop these key words could lead to mistakes by future developers.

13 ?:

Generally avoid this unless the space savings really improves readability. The problem is people often put too much code in the ternary operator. Nested conditional statements should be avoided no matter how "clever" their structure may appear.

14 Preprocessor

14.1 #define CONSTANT Value

The #define preprocessor is used for string substitution by the preprocessor. It can be used for defining constants (e.g. PI 3.1415) or to replace a repeated long string of text with a shorter string. (e.g. SA to replace struct sockaddr)

14.1.1 The Rules

- All defines must be all uppercase with words separated by underscores

When used to define a constant it should not be used where:

- type checking is required (in this case use a const var)
- lists of logical values, use an enumerated list instead

When used for string replacement:

- only declared locally at the start of the block
- undefined at the end of the block

14.2 Macros

14.2.1.1 Reasons For These Rules

Following the guidelines below will ensure that macros are easily differentiated from functions, that their scope is easily determined.

14.2.1.2 The Rules

- Only use macros to in line small pieces of code when overhead of a function call is unacceptable due to size or execution overhead.
- All characters in a macro name must be upper case.
- Words must be delimited by '_'.
_
- Parameters must be wrapped in parenthesis when part of the definition.
- Multi-line macros must be written in such a way that they can be used like functions. If necessary multi-line macros must be wrapped with a set of braces.

14.2.1.2.1 Example

Multi-line Macro

```
#define SWAP_SCREEN_BUFFER() \
```

```
graphics_vsFpgaRegister.usSwapCount++;\  
graphics_vsFpgaRegister.usSwapControl |= GRAPHICS_SWAP_BIT
```

Note: no final ';'.

Macros

```
#define MAX(a,b)    (((a)<(b))?(b):(a))  
#define MIN(a,b)   (((a)<(b))?(a):(b))
```

14.3 #ifdef

14.3.1 Header Files

14.3.1.1 The Rules

- The body of the file should be wrapped inside an `#ifndef..#endif`. These make sure this file is included only once. Use the filename inside of `__` and `_H_`. `__FILENAME_H_` for example.

14.3.2 All Other Places

The preprocessor in C/C++ is very powerful and should be used sparingly. `#ifdef` and `#endif` blocks can (and do) very quickly make an easy to manage program unmanageable. The preprocessor in C is really just a hack to the language and as with all hacks very powerful but come back to bite you. For this reason `#ifdef` blocks should be avoided when ever possible. It is not always possible as in the header file. When compiling designing for multiple hardware architectures consider abstracting the hardware, using common header files but different module files in the project (Makefile).

15 Comments

15.1 Reasons For These Rules

This section is more of a suggestion. These “rules” can be ignored, however it is suggested that you follow them as they will add consistency to the code.

The code tells you how something is happening. **Comments should first explain what is happening and then possibly why.**

15.2 The Rules

- Comments for a section are to appear above the code being described. In the case of a single line of code the comment may appear after the code on the same line as the code.
- Comments on the end of a line should use the // format
- Comments on their own line/mult-line should use /* */
- Header blocks use /* */
- Comments after variables use //

16 Initialization, Compile Time Vs Run Time

If something is known at compile time set it at compile time, don't call a function to discover the value.

If a variable will only ever have one value make it a constant.

If a function pointer is being passed and that pointer only has one value don't use a pointer. The use of function pointers often is a sign that code has been poorly ported from a C++ project.

17 Concurrency

Concurrency occurs when two executions of the code try and access the same resource. Concurrency leads to complicated code, slow performance and difficult to find bugs. Many programmers automatically try and use mutexes and semaphores to arbitrate use of the resource. These should be used as a last resort. Instead the code should be architected to minimize the impact of concurrency.

The best solutions in order are:

- Don't - have only one execution do all the work on the resources or be single threaded
- Atomic operations - some processors allow atomic arithmetic operations
- DMA and other hardware assistance
- Binary flag - often used for signaling, one execution sets the flag the other clears. Often used to limit how fast the clearer runs.
- Priority - The higher priority execution is the only one that can do read/modify/write. Suitable for count down timers.
- Single writer - Only one execution can modify the value. Suitable for circular queues head and tail pointers.
- Disable interrupts - simple and effective if only a very small amount of code needs to be executed
- Yield - use a non-preemptive OS
- Mutex, Semaphore - last resort

18 Miscellaneous

These are rules that didn't fit anywhere else.

18.1 Fall Through Switch Case

In a switch / case block if the case does not end with a break place a comment in the case statement noting that it does not. Ensure the comment is clearly visible.

18.2 File Naming

File names should all be lowercase for interoperability with an OS that shall remain nameless

18.3 Integer Sizes

integer variable types – if the position of bits in a variable are ever used declare the variable of type `u_intNN_t` where NN is the size of the integer in bits. If the number is used strictly for counting declare it of type `int`.

18.4 Leftover Debug Print

Debug printing should be removed once the bug or feature has been fixed or created. Do not leave them in the code. Logging is for diagnosing issues in the field or non-debugging uses. Logging is fine, debug code should be cleaned up.

18.5 Return Statements

It is permissible to have multiple return statements in a function if it makes the overall code more readable. Common times to have more than one return is in similarly structured blocks that start with some sort of mutually exclusive checks.